

Persistence Migration Report

Task Management REST API · In-Memory List → SQLite Database

Track	Backend AI Engineering — Python / FastAPI Lane
Assignment	W3 · A2 — Connecting your CRUD to the Database
Stack	FastAPI · Pydantic · SQLite · DB Browser for SQLite
Author	Goraya · github.com/Goraya0999
Deliverable	Stage 0 – 5 build, verified end to end with screenshots

This report documents the migration of a FastAPI Task API from in-memory storage to a persistent SQLite database, with the same five CRUD endpoints behaving identically before and after the change.

1. Overview

The Task API originally kept its data in a Python list: functional, but every restart wiped the task list clean. This assignment replaces that list with a SQLite database (**tasks.db**) while keeping the API surface completely unchanged — the same five endpoints, the same request and response shapes, the same status codes. Only the storage layer underneath was rebuilt.

This is the core separation the assignment is built around: the API is the promise, and the database is where that promise is kept. Everything documented below is evidence that swapping the storage did not change the promise.

1.1 Architecture

Before	Client → FastAPI routes → Python list (memory)
After	Client → FastAPI routes → SQLite (tasks.db, on disk)

1.2 Endpoint contract (unchanged by the migration)

Method	Path	Behaviour	Status codes
GET	/tasks	List every task (supports ?search=, ?done=)	200
GET	/tasks/{id}	Fetch a single task	200 / 404
POST	/tasks	Create a task	201 / 400
PUT	/tasks/{id}	Update title + done	200 / 400 / 404
DELETE	/tasks/{id}	Remove a task	204 / 404
GET	/stats	Task counts via SQL COUNT()	200

2. Database Schema

A single table, created automatically on first run if it doesn't already exist:

```
CREATE TABLE IF NOT EXISTS tasks (  
  id    INTEGER PRIMARY KEY AUTOINCREMENT,  
  title TEXT NOT NULL,  
  done  BOOLEAN NOT NULL DEFAULT 0  
);
```

Three example tasks are inserted the first time the application runs, wrapped in a transaction so the seed is all-or-nothing. A row count check (`SELECT COUNT( *) FROM tasks`) before inserting is what stops the examples from multiplying on every restart — this is verified in Section 3.

2.1 Parameterized queries

Every query that touches user-supplied data uses a `?` placeholder rather than string concatenation, e.g.:

```
self.cur.execute(  
  "SELECT * FROM tasks WHERE id = ?", (id,) )
```

This is what keeps a task title like `0'Brien's report` (or a deliberately malicious one) from ever being interpreted as SQL.

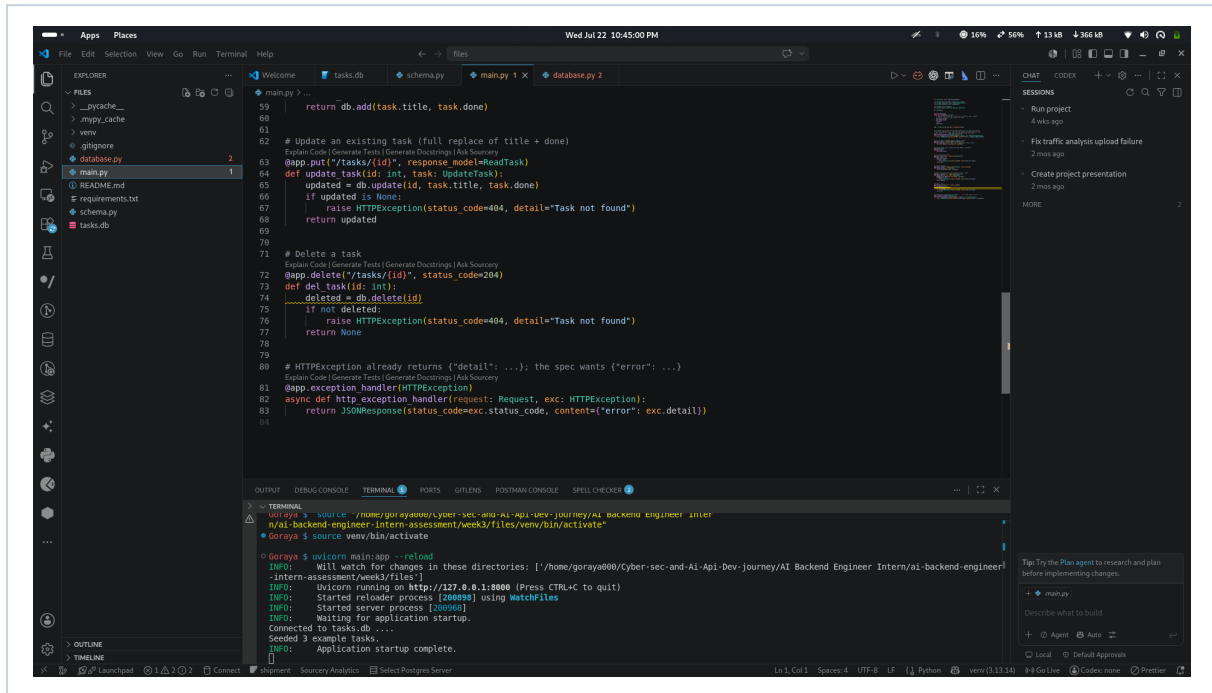
3. Verification Walkthrough

Each checkpoint below is demonstrated live against the running API and the raw database file, side by side.

3.1 Stage 0 — Application startup & auto-seed

On launch, the app connects to `tasks.db` (creating the file if it's missing), creates the `tasks` table if it doesn't exist, and seeds three example rows only when the table is empty.

Figure 1 — Application startup

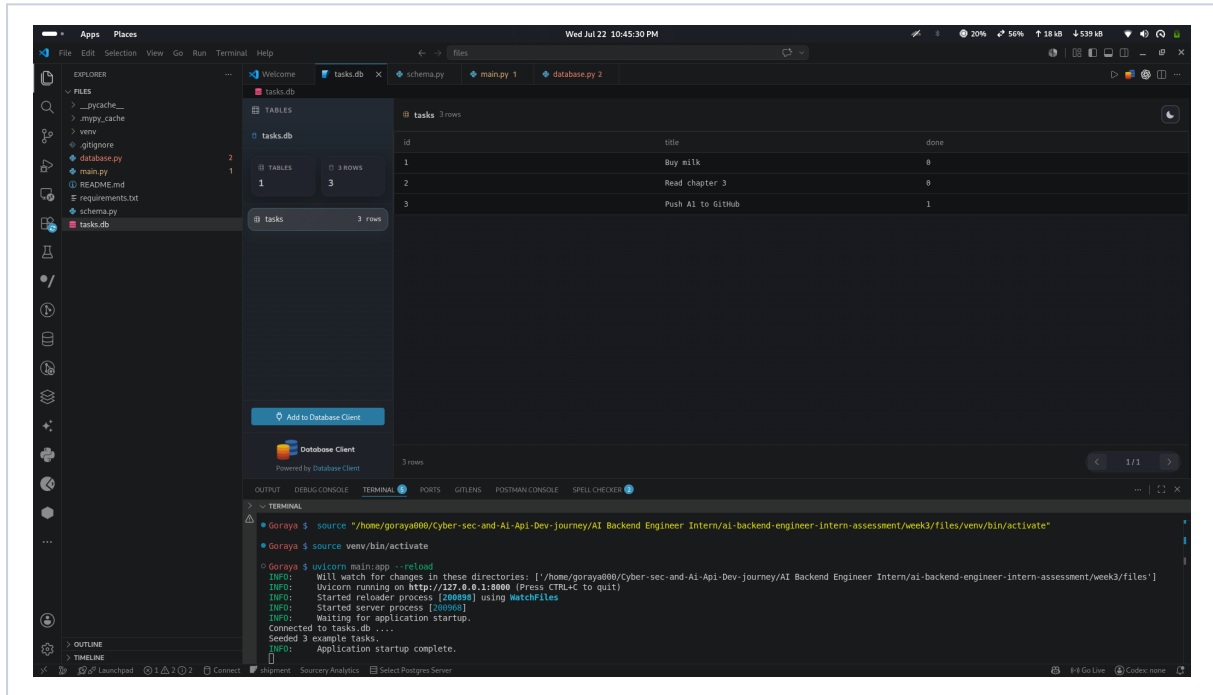


Server boot log: database connection established, table verified/created, and the API started and ready to accept requests.

3.2 Seed data, viewed directly in the database file

Opening `tasks.db` in DB Browser for SQLite confirms the three seeded rows exist on disk — independent of the API, in the actual file.

Figure 2 — Seeded rows in DB Browser for SQLite

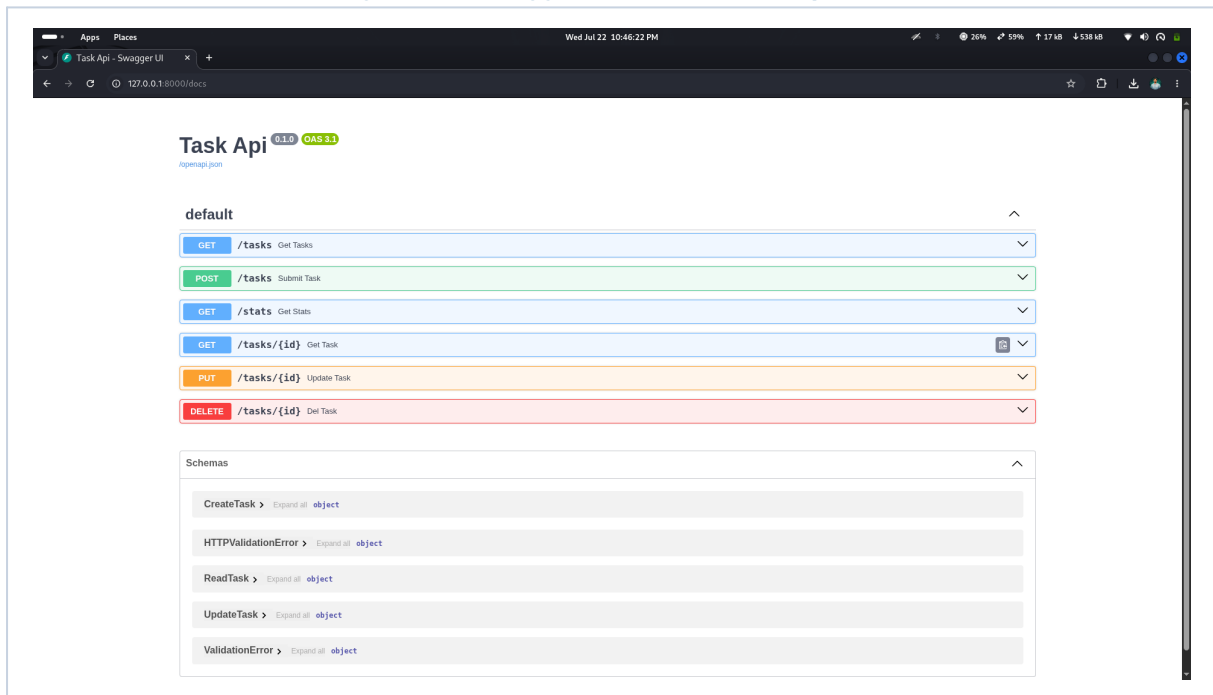


The tasks table after first startup: exactly three rows, matching the seed logic, with no duplication from previous runs.

3.3 Stage 1 — Interactive API documentation

FastAPI's auto-generated Swagger UI (`/docs`) lists all five CRUD endpoints plus the optional `/stats` route, confirming the contract from Section 1.2 is exposed correctly.

Figure 3 — Swagger UI — available endpoints



GET /tasks, GET /tasks/{id}, POST /tasks, PUT /tasks/{id}, DELETE /tasks/{id} and GET /stats, all registered against the SQLite-backed router.

3.4 Stage 1 — Reading with filters

GET /tasks supports optional query parameters that translate directly into SQL WHERE clauses: `?done=true` filters completed tasks, and `/tasks/{id}` fetches a single row by its primary key.

Figure 4 — Filtered read against the database

The image shows a Swagger UI interface for the Task API. The top bar indicates the API is 'Task Api - Swagger UI' and the URL is '127.0.0.1:8000/doc#/default/get_tasks_tasks_get'. The main panel displays the 'GET /tasks' endpoint. Under the 'Parameters' tab, there are two parameters: 'search' (string, query) with a value of '1', and 'done' (boolean, query) with a value of '--'. Below the parameters are 'Execute' and 'Clear' buttons. The 'Responses' tab shows a '200' status code with a 'Response body' containing a JSON object:

```
{  "id": 1,  "title": "Push AI to GitHub",  "done": true}
```

. The interface also includes a 'Curl' section with the command:

```
curl -X 'GET' \  "http://127.0.0.1:8000/tasks?search=1" \  -H 'accept: application/json'
```

 and a 'Request URL' section with the URL:

```
http://127.0.0.1:8000/tasks?search=1
```

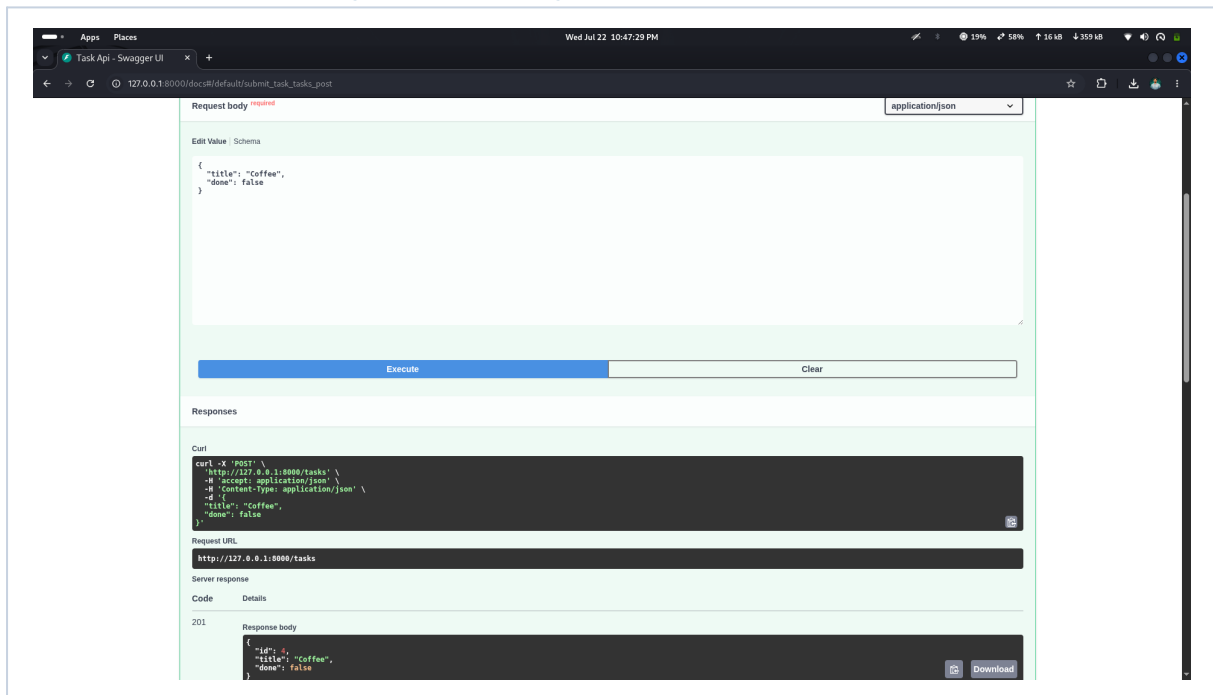
.

Querying tasks by completion status / id and receiving only the matching row(s) back — the filtering happens in SQL, not in Python.

3.5 Stage 2 — Creating a task (persisted write)

POST /tasks runs a parameterized INSERT and returns the new row, including the id SQLite assigned it, with a 201 status code.

Figure 5 — Creating a new task via POST /tasks

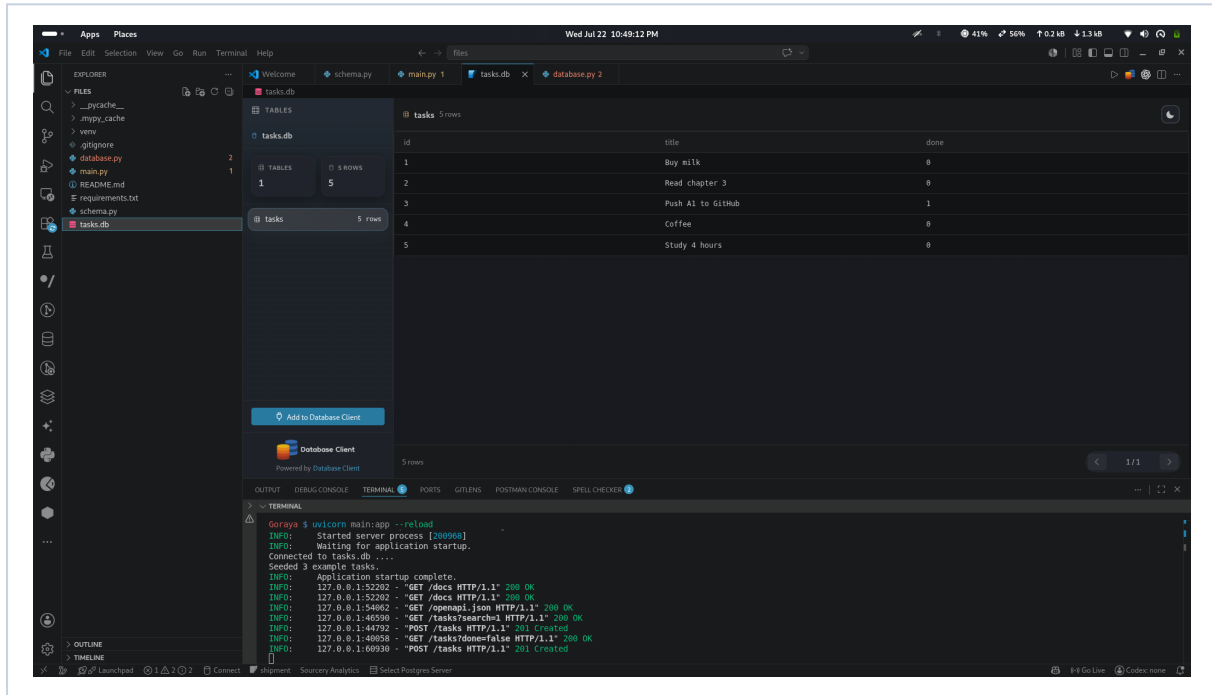


A new task submitted through the API, written straight into tasks.db and echoed back with its database-assigned id.

3.6 Persistence proof — the database file after insertion

Re-opening tasks.db in DB Browser after the request above shows the new row sitting alongside the original three — written by the API, read back by an entirely separate tool, with no restart in between.

Figure 6 — Database file after the insert



The tasks table now contains the seeded rows plus the task created via the API in Figure 5 — direct evidence that writes reach disk, not memory.

4. Manual SQL — Talking to the Database Directly

With DB Browser for SQLite open on `tasks.db`, the following queries were run by hand in the "Execute SQL" tab to confirm the API and the raw file are, in fact, the exact same source of truth:

```
SELECT * FROM tasks; -- list every task
SELECT * FROM tasks WHERE done = 1; -- only completed tasks
SELECT COUNT(*) FROM tasks; -- how many tasks exist
```

Calling `GET /tasks` immediately after each change reflected it instantly through the API — there is no sync step, because there is only one file being read by both tools.

4.1 Why SQLite

- Zero setup: no separate database server process, no connection string, no Docker container.
- Single file (`tasks.db`): trivial to inspect, back up, version, or delete to reset.
- The exact same SQL and the same `?` parameter style carry over unchanged to Postgres or MySQL later — only the driver/connection changes.